Algorithm

1. Select the last element as pivot.
2. Partition the array around the pivot.
3. Count every comparison with the pivot.
4. Recursively sort the left and right partitions.
5. Display the sorted array and comparison count.

C++ Program

```cpp
#include <iostream>
#include <vector>
using namespace std;

long long comparisons = 0;

int partitionArray(vector<int>& a, int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        comparisons++;

        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }

    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(vector<int>& a, int low, int high) {
    if (low < high) {
        int p = partitionArray(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    int n;
    cin >> n;
```

```
    vector<int> a(n);

    for (int i = 0; i < n; i++)
        cin >> a[i];

    comparisons = 0;

    quickSort(a, 0, n - 1);

    cout << "Sorted Array : ";
    for (int x : a)
        cout << x << " ";

    cout << "\nComparisons : " << comparisons << endl;

    return 0;
}
```

Sample Input

```
7
38 27 43 3 9 82 10
```

Sample Output

```
Sorted Array : 3 9 10 27 38 43 82
Comparisons : 15
```

Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: $O(n^2)$
- Auxiliary Space: O(log n) average recursion